

掀起 AI 编程智能体的引擎盖 v3.4

v3.4 变更说明： 重写 3.2 节 Orchestrator-Worker 架构为完整版（含协调原语工具包、Team 5 阶段生命周期、通信机制表、5 系统架构对比表）。新增独立协议参考文档 [claude-message-protocol.md](#)，覆盖 content block 类型、tool_use 协议、并行调用、Team 消息格式、三方协议对比，课程 1.2/3.2/5.2a 节均引用。

v3.3 变更说明： 重写 5.2 节为完整 MVP 架构说明（含 9 个工具、71 项测试），新增 5.2a 协议适配层架构决策（Claude tool_use ↔ Qwen 原生格式双向转换、格式对比表、设计收益分析）。

v3.0 变更说明： 新增实践演练方案（第五章），包含 Demo 模型选型分析、各模块 Demo 策略设计、MVP 代码库集成方案。基于 v2.x 全部内容。

v2.0 变更说明： 基于 Claude Code (Opus 4.6) 对 v1.0 的技术审校，修正了若干与 Claude Code 实际实现不符的描述，重写了模块三（Agent Team），新增了安全与权限控制章节。原始 v1.0 由 Gemini 协助生成。

一、论文阅读清单

按照 认知觉醒 → 确立战场 → 构建系统 → 架构反思 的认知递进排列：

第一阶段：认知觉醒（解决“如何让模型动起来”的问题）

1. ReAct (智能体范式的鼻祖) * 标题： ReAct: Synergizing Reasoning and Acting in Language Models * 作者： Shunyu Yao, et al. (Princeton University, Google Brain) * 时间： 2022

2. Toolformer (工具调用的底层原理) * 标题： Toolformer: Language Models Can Teach Themselves to Use Tools * 作者： Timo Schick, et al. (Meta AI) * 时间： 2023

3. Reflexion (自我反思与纠错机制) * 标题： Reflexion: Language Agents with Verbal Reinforcement Learning * 作者： Noah Shinn, et al. (Northeastern University, MIT) * 时间： 2023

第二阶段：确立战场（解决“如何评价 AI 程序员”的问题）

4. SWE-bench (行业公认的金标准测试集) * 标题: SWE-bench: Can Language Models Resolve Real-World GitHub Issues? * 作者: Carlos E. Jimenez, et al. (Princeton University) * 时间: 2023

第三阶段：构建系统（解决“工程上如何复现 Claude Code”的问题）

5. SWE-agent (ACI 接口设计的教科书) * 标题: SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering * 作者: John Yang, et al. (Princeton University) * 时间: 2024

6. AutoCodeRover (基于 AST 结构化检索的先驱) * 标题: AutoCodeRover: Autonomous Program Improvement * 作者: Yuntong Zhang, et al. (National University of Singapore) * 时间: 2024

7. CodeAct (让模型直接执行 Bash/Python 的理念) * 标题: Executable Code Actions Elicit Better LLM Agents * 作者: Xingyao Wang, et al. (UIUC, Google DeepMind) * 时间: 2024

第四阶段：架构反思（解决“如何让系统更稳健、更低成本”的问题）

8. Agentless (回归简洁流水线的深度思考) * 标题: Agentless: Demystifying LLM-based Software Engineering Agents * 作者: Chunqiu Steven Xia, et al. (UIUC) * 时间: 2024

补充阅读：多智能体协同（模块三参考）

9. MetaGPT (多角色 SOP 协同) * 标题: MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework * 作者: Sirui Hong, et al. * 时间: 2023 * 定位: 学术参考，了解固定角色协同的思路。注意: Claude Code 的 Team 模式并未采用此方案。

10. AutoGen (对话驱动的多智能体框架) * 标题: AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation * 作者: Qingyun Wu, et al. (Microsoft Research) * 时间: 2023 * 定位: 学术参考，了解对话式协同的通用框架设计。

11. OpenHands (工业级开源实现) * 标题： OpenHands: An Open Platform for AI Software Developers as Generalist Agents * 作者： Xingyao Wang, et al. * 时间： 2025 * 定位： 最接近工业实践的开源参考实现，推荐重点阅读其 agent 调度与沙箱设计。

二、论文认知地图：两条主线

这 8+3 篇论文可以沿两条正交的主线来理解。

主线一：任务结构线 —— Localization → Repair → Validation

所有编程 agent（不论架构多复杂）本质上都在做三件事：

找到改哪里（Localization）——> 生成修改（Repair）——> 验证对不对（Validation）

这是 Agentless 论文显式提出的三阶段框架，但它抽象出的是一个**通用任务结构**，同样适用于 Claude Code：

阶段	Agentless 实现	Claude Code 实现
Localization	固定漏斗：文件 → 函数 → 行	动态探索：Glob → Grep → Read → LSP
Repair	多候选采样 + search/replace	单次生成 + Edit tool search/replace
Validation	测试套件筛选最优候选	Bash 跑测试，失败则回溯到 L 或 R

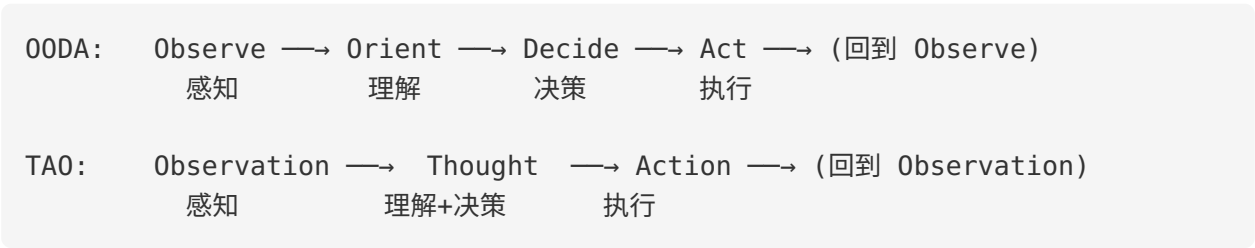
区别只是 Agentless 的三阶段是**单向流水线**（不可回溯，靠广度采样兜底），而 Claude Code 的三阶段是**可回溯的循环**（靠深度迭代兜底）。骨架是同一个。

主线二：OODA 演进线 —— 每篇论文优化了循环的哪个环节

OODA 与 TAO 的关系

OODA（Observe → Orient → Decide → Act）是军事战略家 John Boyd 提出的通用决策循环框架。ReAct 论文提出的 TAO（Thought → Action → Observation）是 OODA 在

LLM agent 领域的一个**特化投影**，将 Orient（理解）和 Decide（决策）压缩为一个 Thought 步骤——因为 LLM 在一次推理中天然地同时完成理解和决策：



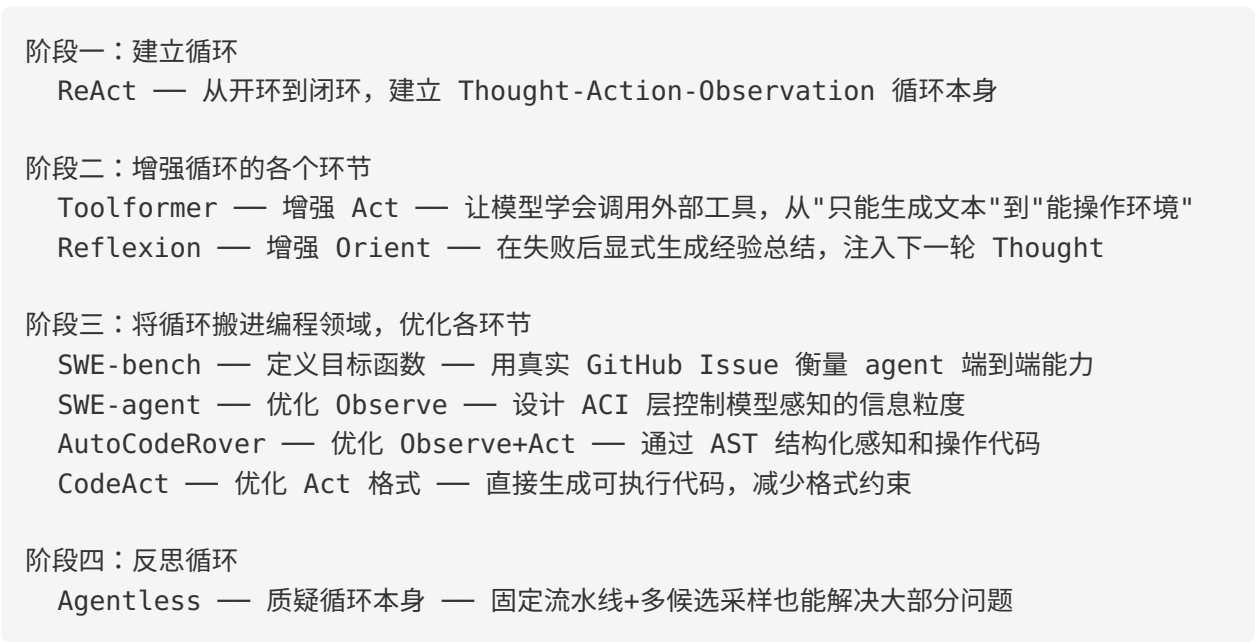
Boyd 的核心洞察：循环转速决定胜负

Boyd 的理论源自朝鲜战争空战。美军 F-86 对苏制 MiG-15 的击杀比高达 10:1，但 MiG-15 在纸面参数上全面占优（爬升更快、升限更高）。F-86 赢在两个“不起眼”的设计：气泡式座舱（360° 视野 = Observe 更快）和液压飞控（轻推即响应 = Act 更快）。

Boyd 由此得出的核心洞察是：**在动态环境中，快速的“足够好”决策胜过迟到的“完美”决策——因为完美决策做出时，它所针对的局面可能已经不存在了。** 循环更快的一方可以持续用最新信息纠偏，更慢的一方则在对过时的现实做反应——几轮下来，慢的一方彻底失去态势感知。

映射到 AI agent：模型的基础能力是门槛，但循环机制是放大器——一个中等能力的模型跑多轮 ReAct 迭代，往往胜过更强的模型做一次性开环生成。这正是 ReAct 胜过开环生成的根本原因。

8 篇论文在 OODA 循环上的演进



每篇论文在 OODA 环节上的精确定位：

论文	优化的 OODA 环节	核心贡献
ReAct	建立循环本身	革命性：将 LLM 从开环生成带入闭环控制
Toolformer	Act（行动能力）	让模型学会调用外部工具，解决“手”的问题
Reflexion	Orient（理解反馈）	将失败信号转化为结构化经验总结，解决“从错误中学习”的问题
SWE-bench	（定义战场）	建立编程领域的标准评测基准
SWE-agent	Observe（感知质量）	设计 ACI 控制信息粒度，解决“眼睛”的问题
AutoCodeRover	Observe + Act	通过 AST 给模型装上“代码 X 光”，结构化感知 + 操作
CodeAct	Act（行动格式）	用可执行代码替代 JSON 工具调用，简化动作表达
Agentless	质疑循环本身	证明流水线也能达到 agent 70-80% 的效果，倒逼 agent 证明额外复杂度的价值

Reflexion 的双层循环模型

Reflexion 在 TAO 循环的基础上增加了一个**外层循环**。内层是标准的 TAO（一个 Episode 内的多步操作），外层是 Episode 之间的学习循环：

外层循环（Reflexion 层，跨 Episode）：

Episode 1: [T → A → O → T → A → O → ...] → 失败 → 生成 Reflection

↓

Episode 2: [T(+Reflection) → A → O → ...] → 失败 → 生成 Reflection

↓

Episode 3: [T(+Reflections 1&2) → A → O → ...] → 成功 → 结束

触发 Reflection 的信号是**失败**（负反馈），但 Reflection 的输出是结构化的经验总结（“我错在哪、为什么错、下次应该怎么做”），这段总结作为**正向指导**注入下一轮 Thought。所以 Reflexion 本质上是一个**负反馈到正向指导的转化器**。

在 Claude Code 的实际实现中，这两层循环被摊平为一个连续对话：stderr 直接进入上下文，模型下一个 Thought 自然参考。但 Reflexion 论文的贡献在于证明了**显式生成经验总结**比“把错误信息扔进上下文让模型自己领悟”更有效。

两条主线的交汇：二维认知地图

两条主线构成一个二维空间：横轴是任务结构（L → R → V），纵轴是 OODA 环节。11 篇论文散布在这个空间中：

	Localization	Repair	Validation
Observe	SWE-agent AutoCodeRover		
Orient	Reflexion		
Act	Toolformer	CodeAct Agentless*	
全循环	ReAct（建立 T-A-O 循环本身）		
评测	SWE-bench（定义端到端评测基准）		
多Agent	MetaGPT / AutoGen / OpenHands		

* Agentless 同时覆盖 L-R-V 三阶段，但作为固定流水线而非循环

这张图就是课程开场“论文全景图”要展示的内容——让听众在第一时间拿到认知地图，后续每个模块都能定位到图上的具体区域。

三、论文够不够？

对于前两个议题（修 Bug、写 Feature）绝对够了，这 8 篇覆盖了单体智能（Single Agent）的核心理论。

对于“Agent Team（多智能体协同）”不够。上述 8 篇主要讲模型如何与“环境（Terminal）”交互，没有讲模型如何与“其他模型（Peers）”交互。补充的 3 篇（MetaGPT、AutoGen、OpenHands）提供了理论底座。

v2.0 额外建议：多智能体部分，学术论文与工业实现的差距很大。建议同时参考 Anthropic 官方的 agent 文档和 Claude Code 的实际行为，而非仅依赖论文。

四、3小时高阶课程大纲

课程名称：《掀开 AI 程序员的引擎盖：Claude Code 与 AI Agent 的底层架构与控制流》

课程受众：高级研发工程师、架构师、效能团队 课程时长：3 小时（每个模块 45 分钟讲解 + 15 分钟 QA 互动）

模块一：破除“文本接龙”迷信 —— 智能体控制论与 Bug 修复原理 (1小时)

场景目标：剖析 `cc` 是如何在完全未知的代码库中，通过试错修复一个 Bug 的。核心引用论文：ReAct, Toolformer, Reflexion

1.1 从开环生成到闭环控制：OODA 与 TAO

- **传统大模型的死穴**：开环生成（Open-loop）与错误累积（Error Accumulation）。一次性生成代码补丁，没有验证环节，错误无从纠正。
- **Boyd 的启示**：朝鲜战争中 F-86 以 10:1 击杀比碾压纸面参数更优的 MiG-15，靠的不是更强的火力，而是更快的决策-反馈循环（OODA: Observe → Orient → Decide → Act）。在动态环境中，快速的“足够好”决策胜过迟到的“完美”决策——因为完美决策做出来的时候，它所针对的局面可能已经不存在了。映射到 AI agent：模型基础能力是门槛，循环机制是放大器。
- **ReAct 范式（ReAct：从开环到闭环的范式革命）**：交替执行“推理（Thought）→ 动作（Action）→ 观察（Observation）”，形成闭环。TAO 是 OODA 在 LLM agent 领域的特化投影——将 Orient（理解）和 Decide（决策）压缩为一个 Thought 步骤，因为 LLM 在一次推理中天然地同时完成理解和决策。
- **两条主线的起点**：ReAct 同时开启了本课程的两条主线——任务结构线（Localization → Repair → Validation 的循环执行）和 OODA 演进线（后续论文分别优化循环的各个环节）。

1.2 工具调用的真实机制：API 层协议而非 Tokenizer 魔法

参考文档：完整的 Claude Messages API 协议格式（content block 类型、tool_use/tool_result 交互流程、stop_reason 信令、并行工具调用、Team 模

式消息格式、与 OpenAI/Qwen 的协议对比) 见 [claude-message-protocol.md](#)。本节讲“为什么”，参考文档讲“是什么”。

- **工具调用的本质 (Toolformer: 赋予模型工具使用能力)**：模型通过 API 的 `tool_use` 协议返回结构化的工具调用请求 (JSON schema)，客户端解析后执行对应操作，再将结果作为 `tool_result` 回传。这是**应用层协议**，不是 tokenizer 层的 special token hack。
- **常见误区澄清**：网上流传的“special token 劫持”说法具有误导性。实际的 Claude API 工具调用流程是：模型输出 → API 返回 `tool_use content block` → 客户端执行 → 将结果封装为 `tool_result` 回传。整个过程在应用层完成。
- **Claude Code 的工具集设计**：`cc` 并非让模型直接输出裸 bash 命令（这是 CodeAct/OpenHands 的路线），而是提供了**结构化的专用工具集** (Read、Edit、Grep、Glob、Bash、Agent...)，每个工具有明确的参数 schema。这降低了格式出错的概率，也让权限控制成为可能。
- **对比 (CodeAct: 代码即动作)**：CodeAct 论文主张让模型直接生成可执行代码作为动作，减少格式约束。这是另一条可行路线 (OpenHands 采用)，各有优劣。Claude Code 选择了结构化工具调用 + 丰富工具集的路线。

1.2a Action Space 设计：少即是多

各论文的 Action Space 看似简单，实则是反复试错后的极简完备集：

系统	Action Space 规模	设计哲学
SWE-agent	~12 个命令 (open, scroll, search_file, edit...)	模拟人类在编辑器中的最小操作集
AutoCodeRover	AST 工具 (search_class, search_method...)	按代码结构而非文本组织动作
CodeAct	仅 Python + Bash	极致简化：代码本身就是动作语言
Claude Code	~10 个工具 (Read, Edit, Glob, Grep, Bash, LSP, Agent...)	每个工具职责单一，参数 schema 严格约束

共同的设计原则：**动作空间越精简，模型选错工具的概率越低；参数 schema 越严格，格式出错的概率越低**。这对应 OODA 的 Act 环节——关键不是“能做的事情越多越好”，而是“精简的动作空间降低决策复杂度，从而提升循环转速”。

每个系统都在精简性与表达力之间做取舍：CodeAct 走极简路线（只要会写代码就什么都能做），但牺牲了权限控制和格式安全；Claude Code 多做了一层抽象（每个工具有明确 schema），换来了可审计性和鲁棒性。

1.3 真实演练剖析：cc 的修 Bug 状态机

- **触发失败**：运行 `pytest` 看到满屏红字。
 - **多轮上下文纠错（Reflexion：化失败为经验）**：模型将 `stderr` 输出纳入对话上下文，在下一轮推理中参考错误信息进行修正。Reflexion 论文揭示了这个过程的本質——它在 TAO 循环外部套了一层**经验积累循环**：失败触发 Reflection（负反馈），Reflection 生成结构化经验总结（正向指导），总结注入下一轮 Thought。在 Claude Code 的实际实现中，这两层循环被摊平为连续对话，但“显式总结经验”比“把错误信息扔进上下文让模型自己领悟”更有效。
 - **防死循环机制（Circuit Breaker）**：模型可能陷入“重复执行同一个错误命令”的怪圈。用 Boyd 的框架理解：这是 OODA 循环卡死——模型的 Orient 环节失灵，无法从重复的 Observation 中提取新信息。工程上通过 max retries、上下文压缩（context compaction）、以及在 system prompt 中注入“如果连续失败请换一种方法”的指令来强制打破循环。
-

模块二：突破上下文重力法则 —— ACI 协议与新功能开发 (1小时)

场景目标：剖析 `cc` 是如何在百万行代码中不迷失，并从零实现一个跨文件 Feature 的。**核心引用论文**：SWE-bench, SWE-agent, AutoCodeRover, Agentless

2.1 为什么传统的 RAG 检索在编程中常常失效？〔SWE-bench：AI 程序员的试金石〕

- **词汇鸿沟与依赖幽灵**：Issue 描述用自然语言，代码用变量名和函数签名，BM25/向量检索难以跨越这道语义鸿沟，找不到真实的 Bug 发生地。
- **AST 与代码感知（AutoCodeRover：结构化代码感知）**：`cc` 需要具备跳转定义（Go to Definition）、查看类签名（LSP hover）的能力，通过代码结构而非纯文本相似度来定位目标。这比把整个文件塞给模型高效得多。

2.2 核心科技：ACI（智能体-计算机接口）设计哲学〔SWE-agent：信息粒度的精确控制〕

- **不是裸 Terminal，而是精心设计的工具层：** `cc` 在模型与操作系统之间插入了一层精心设计的工具接口，控制信息的粒度和格式。
- **防爆窗机制：** 为什么 `cc` 的 Read 工具支持 `offset + limit` 参数？为什么 Grep 有 `head_limit`？因为必须保护 Token Budget——一个 `cat` 大文件就能把整个上下文窗口撑爆。这是 SWE-agent 论文中“粗细粒度漏斗策略（Coarse-to-fine）”的工程化实现。
- **安全的写入协议：** 让模型输出 Unified Diff (`@@ -10,5 +10,6 @@`) 极易出错（行号偏移、上下文不匹配），根本原因在于 diff 依赖行号定位，而模型对数字天然不敏感。业界的共识趋势是转向 **search/replace 范式（内容定位）**——用代码原文本身做锚点，而非行号。Agentless 自创的 `<<<< SEARCH / REPLACE >>>>` 格式和 Claude Code 的 `old_string → new_string` 在精神上完全一致，都属于这一范式，只是各自独立得出了相同结论。Claude Code 在此基础上增加了一层硬约束：如果 `old_string` 在文件中匹配到多处，直接拒绝执行，强制模型提供更多上下文来消歧——宁可报错重试，也不悄悄改错地方。

2.3 真实演练剖析：`cc` 的 Feature 建造流

- **探索期：** `Glob`（按模式找文件）→ `Grep`（按内容搜代码）→ `Read`（精读关键文件）→ `LSP`（跳转定义、查引用）。
- **修改期：** `Edit`（精确替换）→ `Bash` 运行 lint/test → 根据结果迭代修正。

2.4 架构反思：Agentless 三阶段流水线 vs Agent 动态循环〔Agentless：最简实现的力量〕

Agentless 论文提出了一条与 ReAct Agent 截然不同的技术路线——**固定三阶段流水线**：

Localization（定位）→ Repair（修复）→ Patch Validation（验证）

阶段一：分层定位漏斗（Localization）

Localization 内部还有一个层级细化过程：**文件级 → 类/函数级 → 行级**。每一层只给模型看“刚好够做决策”的信息量——先看文件列表选文件，再看代码骨架（skeleton）选函数，最后看具体代码选行号。这个粗到细（Coarse-to-fine）的信息控制策略是全文最具工程洞察力的设计，与 Claude Code 的实际行为（`Glob` → `Grep` → `Read`）高度同构，区别在于 agent 是在 ReAct 循环中动态执行的，而 Agentless 是预设好的固定流水线。

阶段二：多候选采样修复（Repair）

Agentless 在修复阶段不是只生成一个 patch，而是用高 temperature 采样生成多个候选 patch。这暗含一个关键洞察：模型单次生成正确修复的概率可能只有 20%，但生成 10 个候选中有一个正确的概率可以到 80%+。这是用**推理时计算（inference-time compute）换准确率**的策略，与 best-of-N 采样、self-consistency 同源。Agent 方案则是通过多轮试错达到类似效果——跑测试、看报错、再改，本质上也是在搜索正确解，只是搜索策略不同。

阶段三：测试驱动验证（Patch Validation）

用测试套件对所有候选 patch 做筛选，选出通过率最高的作为最终输出。

两条路线的核心对比：

需要注意的是，Agentless 和 Claude Code 在**编辑格式**上殊途同归——都采用了 search/replace 范式（用内容定位，而非行号），真正的对立面是 unified diff（行号定位）。它们的根本差异不在编辑格式，而在**整体架构和容错策略**：

	Agentless（流水线）	Agent（Claude Code）
编辑格式	search/replace（自创语法）	search/replace (old_string → new_string)
唯一性校验	无，生成后直接 apply	有，old_string 不唯一则拒绝执行
搜索策略	并行采样 + 测试筛选	串行试错 + 反馈修正
容错机制	多候选中选一个能过测试的	严格校验 + 模型看到错误后多轮重试
优势	无状态，可大规模并行，成本可控	能利用中间反馈，适应意外情况
劣势	不可回溯，定位错则全盘皆输	可能陷入死循环，上下文会膨胀

关键局限：固定流水线的“脆性”。Agentless 的三阶段严格串行、不可回溯。如果 Localization 阶段定位错了文件，后续 Repair 和 Validation 全部白费。而 Agent 在实际工作中可以根据中间观察动态调整搜索方向（发现改错地方 → 退回重新定位），这种动态纠偏能力是 ReAct agent 的核心优势。

工程权衡结论：Agentless 用简单流水线达到了 agent 方案约 70-80% 的效果，证明了大量问题不需要复杂的 agent 循环。正确的选择不是非此即彼，而是**按任务难度匹配策略**：

定义清晰、范围有限的问题适合流水线（快、便宜、可并行）；需要跨文件追踪、动态探索的复杂任务适合 agent（慢、贵、但能处理意外）。

模块三：从单兵到正规军 —— Agent Team 模式与协同编排 (1小时)

场景目标：剖析 `cc` 的 Agent Team 模式是如何实际工作的：一个主 agent 规划拆解，spawn 子 agent 并行执行，然后汇总交付。**核心引用论文：**MetaGPT (学术对比), AutoGen (学术对比), OpenHands (工业参考) **核心参考：**Claude Code 实际行为与 Anthropic Agent 文档

3.1 为什么我们需要 Agent Team？（物理极限与分治策略）

- **上下文污染 (Context Pollution)：**单个 Agent 在执行了几十步操作后，对话历史膨胀，早期的关键信息被稀释甚至被压缩丢弃，导致模型“迷失”。
- **分治法的天然适配：**软件工程任务本身就是可分解的（前端/后端/测试/文档），将子任务分发给独立的 agent，每个 agent 拥有干净的上下文，效果远好于一个 agent 扛所有。
- **与学术方案的区别 [MetaGPT：固定角色与 SOP 流程]：**MetaGPT 预定义了 PM、Architect、SWE 等固定角色和 SOP 流程。Claude Code 的实际做法更灵活——主 agent 根据任务动态决定要 spawn 几个子 agent、每个做什么，没有固定角色约束。

3.2 Claude Code 的 Orchestrator-Worker 架构

协议基础：Team 模式的并行 spawn、SendMessage 通信、teammate-message 包裹等机制均建立在 Claude Messages API 的 content block 协议之上。协议细节见 [claude-message-protocol.md](#) 第 4-5 节。

核心设计哲学：精华不在某个单一工具，而在于把所有协调原语都做成工具，让 LLM 自己编排。没有硬编码的编排流程、没有固定的 DAG、没有 controller 派发——LLM 在运行时自主决定 spawn 几个 worker、给什么 prompt、如何聚合结果。这与 OpenHands 的 controller 硬编码派发模式形成鲜明对比。

协调原语工具包：

工具	作用	类比
TeamCreate / TeamDelete	创建/销毁协作上下文	<code>docker-compose up/down</code>
TaskCreate + blockedBy	分解任务 + 依赖图	Makefile DAG
Agent (可并行 $\times N$)	Spawn worker	<code>fork()</code>
SendMessage	Agent 间双向通信	IPC / message queue
TaskUpdate	状态同步	共享内存
文件系统	结果传递	管道/共享存储

完整 Team 生命周期（5 阶段）：

阶段 1 · 团队创建 → TeamCreate → TaskCreate($\times N$) → TaskUpdate 设 blockedBy 依赖

阶段 2 · 并行执行 → 单条消息并行调 Agent($\times N$)，每个 Worker 是独立 API 会话

阶段 3 · 进度汇报 → Worker 通过 SendMessage({to:"team-lead"}) 向 Lead 汇报

阶段 4 · 结果聚合 → Lead 读取各 Worker 写入的文件，汇总交付

阶段 5 · 清理 → TeamDelete 删除团队配置

通信机制详解：

机制	方向	实现方式
任务下发	Lead → Worker	Agent 工具的 prompt 参数，包裹在 <code><teammate-message></code> 中
进度汇报	Worker → Lead	<code>SendMessage({to: "team-lead"})</code>
广播	Any → All	<code>SendMessage({to: "*"})</code> （昂贵，慎用）
结果传递	Worker → Lead	文件系统（Worker 写文件，Lead 读文件）
关闭协议	Lead → Worker	<code>SendMessage({message: {type: "shutdown_request"}})</code>

与竞品的架构对比：

系统	谁来编排	delegation 机制	通信模式	上下文隔离
Claude Code	LLM 自主编排	工具调用 (Agent tool)	双向 SendMessage	

系统	谁来编排	delegation 机制	通信模式	上下文隔离
				完全隔离（独立 context window）
OpenHands	Controller 硬编码派发	AgentDelegateAction 事件	共享事件流	部分隔离
SWE-agent	单 agent，无编排	N/A	N/A	N/A
Aider	固定两阶段流水线	Architect → Editor	串行传递	串行隔离
Devin	硬编码角色流水线	专有编排器	沙箱内通信	沙箱隔离

- **Agent Tool 机制：** 主 agent 通过调用 Agent 工具 spawn 子 agent，传入一段自然语言 prompt 描述任务。子 agent 在独立的对话上下文中执行，完成后返回结果摘要给主 agent。
- **任务追踪系统：** 主 agent 使用 TaskCreate / TaskUpdate / TaskList 管理任务状态（pending → in_progress → completed），实现类似看板（Kanban）的协调。这是自然语言驱动的调度，而非预先生成的 DAG。
- **并发执行：** 主 agent 可以在一次响应中同时发起多个 Agent 调用，实现真正的并行执行。独立的子任务并发跑，有依赖关系的串行等待。
- **对比〔AutoGen：对话驱动的多智能体框架〕：** AutoGen 的“对话驱动”多 agent 模式让 agent 之间直接对话。Claude Code 选择了更简单的星型拓扑——所有子 agent 只与主 agent 通信，不互相对话，降低了协调复杂度。

3.3 隔离与共享：子 agent 的执行环境

- **对话隔离，环境共享：** 子 agent 的对话历史是独立的（干净上下文），但继承了工作目录、git 状态、CLAUDE.md 项目指令等环境上下文。这意味着子 agent 能直接读写项目文件。
- **Git Worktree 隔离模式〔OpenHands：沙箱隔离实践〕：** 对于可能产生文件冲突的并行任务，可以设置 isolation: "worktree"，让子 agent 在独立的 git worktree 中工作。每个 worktree 是一个独立的文件系统副本 + 独立分支，从根本上避免了并发写冲突。
- **冲突解决：** 当多个子 agent 在各自 worktree 完成工作后，主 agent 负责 review 和 merge。如果有冲突，主 agent 可以手动解决或再 spawn 一个子 agent 处理。

3.4 Team 模式下的双主线：嵌套的 OODA 与 L-R-V

多智能体论文（MetaGPT、AutoGen、OpenHands）并未显式提出类似 OODA 或 L-R-V 的统一框架，但从 Claude Code 的实际行为中可以投影出嵌套结构：

L-R-V 在 Team 模式下变成分层执行：

Meta-Localization：主 agent 分析需求，识别需要修改的子系统，拆解为子任务
↳ 每个子 agent 在自己的范围内独立执行完整的 L → R → V 循环
Meta-Repair：并发分发 worker，各自执行
Meta-Validation：worker 完成后，主 agent 执行集成测试

OODA 在 Team 模式下变成嵌套循环：

主 agent 的 OODA（慢循环，粗粒度决策）：
Observe：接收各 worker 的完成报告
Orient：判断整体进度，识别阻塞点
Decide：决定是否追加 worker、重新分配任务
Act：spawn 新 worker 或执行 merge

子 agent 的 OODA（快循环，细粒度决策）：
标准的 T → A → O 单体循环，在独立上下文中执行

Boyd 将 OODA 从单机空战扩展到编队作战时的核心发现：**编队的整体 OODA 转速取决于协调开销**。Claude Code 选择星型拓扑（子 agent 不互相通信）正是在最小化协调开销，保持整体循环速度。

3.5 真实演练剖析：一次完整的 Team 级交付

- 用户输入复杂需求 → 主 agent 分析并拆解为若干子任务（TaskCreate）→ 并发 spawn 多个子 agent（Agent tool，可选 worktree 隔离）→ 各子 agent 在独立上下文中完成代码修改 → 返回结果摘要给主 agent → 主 agent review、合并、运行集成测试 → 标记任务完成 → 向用户汇报交付结果。

模块四（加餐）：安全与权限控制 —— 给 AI 装上刹车 (15 分钟)

场景目标：高阶开发者最关心的问题——如何确保 AI agent 不会搞出灾难？

4.1 权限分级机制

- **工具级权限控制：** 每个工具调用都经过权限检查，用户可以配置自动允许的操作范围（如“允许读文件但写文件需确认”）。
- **破坏性操作保护：** `git push --force`、`rm -rf`、删除分支等高风险操作即使在 agent 自主模式下也需要用户确认。

4.2 沙箱与 Hook 机制

- **Bash 沙箱：** 命令执行有超时限制（默认 2 分钟），防止 agent 启动失控的长时间进程。
- **Hook 系统：** 用户可以配置在特定事件（如工具调用前/后）自动执行的 shell 命令，实现自定义的安全策略和自动化流程。

4.3 CLAUDE.md：人类对 AI 的“宪法”

- 项目根目录的 `CLAUDE.md` 文件作为 agent 的行为准则，每次对话都会加载。这是人类控制 agent 行为边界的关键机制——相当于用自然语言写的“宪法”。

五、实践演练方案

5.1 Demo 模型选型：Qwen2.5-Coder-7B

候选对比

维度	Qwen2.5-Coder-7B (Dense)	Qwen3-Coder-32B-A3B (MoE)
架构	Dense 7B	MoE 32B 总参 / ~3B 激活
每 token 推理计算量	7B	~3B
知识容量	7B	32B（专家分散存储）
推理速度	基准	更快（激活参数更少）
结构化输出可靠性	Dense 架构对 JSON schema 格式更稳定	MoE 路由机制引入格式不一致风险

Dense vs MoE：结构化输出可靠性的技术原理

Dense 架构的格式稳定性来源。 Dense 模型的每一个 token 都经过**全部参数**处理。生成 JSON 时，模型需要同时追踪两件事：语义层（该调用哪个工具、传什么参数）和格式层（当前在 { 内部还是外面？上一个 key 后面需要 : 还是 , ？引号是否闭合？）。在 Dense 架构下，这两件事由**同一组参数**共同处理，格式状态的传递是连续且确定的。

MoE 架构的三个结构性弱点：

(1) 路由是逐 token 的离散决策。 MoE 每一层有多个 Expert（FFN 子网络），一个 Router 网络为每个 token **独立**选择 top-k 个 Expert 激活。生成 {"tool": "Read", "args": {"path": "/tmp"}} 时，不同 token 被不同 expert 子集处理：

```
token  "{"      → 激活 Expert 2, 5
token  "tool"    → 激活 Expert 1, 3
token  ":"       → 激活 Expert 2, 7
token  "Read"    → 激活 Expert 4, 6
...
```

格式正确性依赖于这些独立路由决策之间的**隐式一致性**——但这种一致性没有显式保障。Dense 模型不存在这个问题，因为每个 token 都走同一组参数。

(2) Router 在置信度低时容易抖动。 Router 本质上是一个小型线性层 + softmax，输出各 expert 的概率分布后取 top-k。当某个 token 处于多个 expert 的决策边界（概率接近）时，微小的输入变化可能选中不同的 expert。对语义 token（"Read", "/tmp"）影响有限，但对**格式关键 token**（{, }, :, ,, "）而言，路由抖动可能把 token 送到一个对 JSON 语法不够“熟练”的 expert——结果就是少了一个逗号或多了一个引号。

(3) 激活参数量直接限制每 token 的推理容量。 3B active vs 7B dense：7B 的每个 token 用全部参数同时维持语义理解 + 格式追踪；3B 只有不到一半的容量做同样的事。生成自然语言时格式宽松，3B 够用；生成 JSON 时**一个字符的错误就是解析失败**，更少的参数意味着更容易在语义和格式之间顾此失彼。

一句话总结： Dense 的优势不是“更聪明”，而是**每个 token 都由同一组完整参数处理，格式状态的传递是确定性的**。MoE 的路由机制让不同 token 经过不同参数子集，格式一致性变成了概率性的——而 JSON 对错误的容忍度是零。

工业界最佳实践：结构化输出的四层防御

在生产环境中，格式可靠性不是靠“选对模型”单点解决的，而是**分层防御**：

第一层：约束解码（Constrained Decoding）——治本。这是工业界真正的杀手锏。在每一步 token 生成时，根据 JSON schema 或 CFG（上下文无关文法）**屏蔽不合法的 token**，模型只能从合法 token 中采样。例如，当模型刚生成了 `"tool":` 后，合法的下一个 token 只有 `"` ——格式正确性从概率问题变成了**确定性保证**。核心工具链包括 XGrammar（接近零开销）、Outlines（最早的开源实现）、lm-format-enforcer 等，vLLM 和 SGLang 均已原生集成。**无论 Dense 还是 MoE，启用约束解码后生成的 JSON 都是语法合法的——MoE 的路由噪声在格式层面被彻底消除。**

第二层：Tool-calling 专项微调。Hermes 系列、Gorilla、Qwen-Agent 等模型通过 function calling SFT 显著提升工具调用准确率。Berkeley Function Calling Leaderboard (BFCL) 的数据表明，**专项微调对 tool calling 准确率的提升远大于单纯增大模型参数。**

第三层：鲁棒解析（本课程 MVP 已实现）。多策略 fallback 解析：XML → 代码块 → 裸 JSON → JSON 修复。这是在模型端不可控时的客户端兜底，MVP 的 `parser.py` 正是此层的实现。

第四层：应用层重试。格式校验 + 失败重试 + temperature 调整 + 降级到更强模型。MVP 的 `client.py` 中“检测到疑似失败的 tool call 时自动要求模型重试”即属于此层。

前沿模型如何跳过这些防御层：Claude 的做法

上述四层防御是为中小模型设计的。前沿模型（Claude Opus / Sonnet）从根本上跳过了前三层：

规模消解了格式-语义冲突。小模型的核心矛盾是参数容量有限，生成 JSON 时格式追踪和语义推理争夺同一池资源。Opus / Sonnet 是前沿规模的 Dense 模型，参数量比 7B 高出一到两个数量级，同时维持 JSON 格式状态和深层语义推理的容量绑定根本不存在——格式正确性是模型能力的自然副产品，不需要额外工程手段来保障。

工具调用是原生能力，不是后期外挂。小模型的 tool calling 通常是在基座模型上做 function calling SFT，本质是“教”一个已训好的模型学新技能。Claude 的工具调用能力在训练过程中深度集成——模型从一开始就在 `tool_use` 场景上训练，生成 `tool_use` content block 和生成自然语言一样自然。

API 层提供结构化协议保障。Claude 的工具调用不是“模型生成一段 JSON 文本，客户端自己解析”。模型内部生成后，API 基础设施将其结构化为带类型的 `tool_use` content block（含 `id`、`name`、`input` 字段），客户端拿到的是已经过验证的结构化对象。对比之下，MVP 的小模型在原始文本流里“挖”tool call，所以需要 `parser.py` 的三策略 fallback。

剩余挑战转移到语义层。 对 Claude 来说，格式正确性已不是问题。工程重心全部在语义层：选对了的工具没有？参数值对不对？调用时机对不对？Claude Code 的 system prompt 设计、工具 schema 约束（如 Edit 的 old_string 唯一性校验）、以及“连续失败请换方法”的指令，都是在语义层做防御。

模型规模与工程重心的关系

模型规模	格式层（语法正确性）	语义层（调用正确性）
小模型（7B）	需要四层防御 + 约束解码	单次推理能力有限，靠 ReAct 反思循环放大
前沿模型（Claude）	规模 + 训练已解决	主战场：prompt 设计、schema 约束、权限控制

值得注意的是，“单次推理能力有限，靠反思循环放大”绝非权宜之计——这恰恰是本课程的核心论点。2026 年的 7B 模型放在 2023 年就是 GPT-3.5 级别的世界一流水准。ReAct 循环让这样一个“中等能力”的模型通过多轮试错-反馈-修正，达到远超其一次性生成能力的效果——**这就是 Boyd 理论的活体验证：基础能力是门槛，但循环机制是真正的放大器。** 一个 7B 模型跑 5 轮 TAO 循环修好一个 bug，比同一个模型一次性开环生成修复 patch 的成功率高出数倍。课程 demo 选用 7B 而非前沿模型，正是要让观众亲眼见证这个放大效应。

课程 Demo 的刻意选择

课程 demo **故意不启用约束解码**。原因是：启用后格式 100% 正确，观众无法看到小模型的原始格式错误。不启用约束解码，观众才能亲眼看到格式崩坏，理解为什么工业界需要上述四层防御。这是一个刻意的教学设计——**先展示裸奔的风险，再讲穿甲衣的必要性。**

选择 Qwen2.5-Coder-7B 的三个理由

理由一：Dense 架构对 tool calling 更可靠。 课程全部 demo 依赖结构化工具调用（<tool_call> XML 标签内嵌 JSON）。MoE 架构在小激活量下格式鲁棒性不如同等规模的 dense 模型。一次 JSON 格式崩坏就能让 live demo 翻车。

理由二：失败本身就是教材。 课程设计原则是“先讲失败再讲兜底”。7B dense 模型在模块二后段和模块三会自然暴露出上下文丢失、死循环、工具调用格式错误等问题——正好是 circuit breaker、context compaction 的活教材。比起一个“差不多能跑但偶尔格

式崩坏”的 MoE，不如用一个“简单任务稳定完成、复杂任务可预期地失败”的 dense 模型，让失败模式可控、可解释。

理由三：教学叙事更顺。 用 7B 演示模块一二，观众看到 ReAct 循环确实能放大中等能力模型的表现（呼应 Boyd 的“基础能力是门槛，循环机制是放大器”）；到模块三 Agent Team 时，7B 的力不从心自然引出“为什么工业级系统需要 Opus/Sonnet 级别模型做 orchestrator”——形成从能力边界到工程选型的认知闭环。

5.2 MVP 代码库：架构与协议适配

课程 demo 基于 `mvp/` 目录下的完整实现，架构如下：

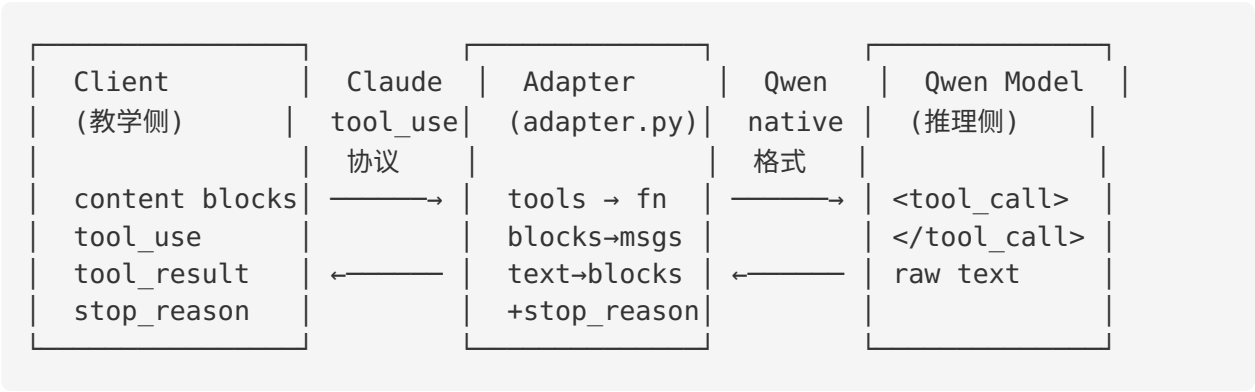
```
mvp/
├── src/
│   ├── model_server.py    ← FastAPI 模型推理服务（加载 Qwen，SSE 流式输出）
│   ├── adapter.py         ← 协议适配层（Claude tool_use ↔ Qwen 原生格式双向转换）
│   ├── client.py          ← Agent 核心循环（ReAct loop，最多 10 轮工具调用）
│   ├── parser.py          ← 工具调用解析器（XML / 代码块 / 裸 JSON 三策略）
│   ├── tools.py           ← 基础工具（Read / Write / Edit / Grep / Bash，5 个）
│   ├── task_tools.py      ← 任务管理工具（TaskCreate / TaskUpdate / TaskList，3 个）
│   ├── agent_tool.py      ← 子 Agent 生成工具（Agent，1 个）
│   └── main.py            ← REPL 入口
├── tests/
│   ├── buggy_code.py      ← 预置 Bug：off-by-one IndexError
│   ├── large_module.py    ← 442 行大文件（含 ZeroDivisionError bug，ACI 演示用）
│   ├── test_all.py        ← 完整测试套件（71 项，覆盖工具/解析/适配/任务/集成）
│   └── ...
```

5.2a 关键架构决策：协议适配层（Adapter Pattern）

协议参考： Claude Messages API 的完整格式规范（content block 类型系统、tool_use/tool_result 交互流程、与 OpenAI/Qwen 的三方对比表）见 [claude-message-protocol.md](#)。本节聚焦“为什么需要适配”和“适配层怎么做”。

问题： 课程 MVP 用 Qwen2.5-Coder-7B 做推理后端，但客户端需要教学生“真实的 Claude Code 工作原理”。如果客户端直接说 Qwen 的原生格式（`<tool_call>` XML 标签 + OpenAI 函数调用风格），那代码和 Claude Code 的真实架构没有任何映射关系，观众学到的是“如何接 Qwen API”而非“Claude Code 是怎么工作的”。

决策： 引入一个**协议适配层**（`adapter.py`），让客户端和 Claude Code 说相同的协议，让模型说自己训练时的原生格式，适配层负责双向翻译：



这个适配层做的事情，本质上就是 Anthropic API 基础设施在做的事——把模型的原始文本输出结构化为 **typed content blocks**。区别在于 Anthropic 用前沿模型 + 约束解码做确定性保证，我们用 7B 模型 + 鲁棒解析做概率性兜底。

协议格式对比（核心教学素材）：

维度	Claude tool_use 协议 (客户端侧)	Qwen 原生格式 (模型侧)
工具定义	<code>input_schema:</code> <code>{type:</code> <code>"object", ...}</code>	<code>type: "function", function:</code> <code>{parameters: ...}</code>
工具调用	<code>{type: "tool_use",</code> <code>id: "toolu_xxx",</code> <code>name: "Read",</code> <code>input: {...}}</code>	<code><tool_call>{"name": "Read", "arguments":</code> <code>{...}}</tool_call></code>
工具结果	<code>{type:</code> <code>"tool_result",</code> <code>tool_use_id:</code> <code>"toolu_xxx",</code> <code>content: "..."}}</code>	<code><tool_response>...</tool_response></code>
ID 关联	用 <code>tool_use_id</code> 精确 关联请求和结果	无 ID，靠位置顺序匹配
停止信号	<code>stop_reason:</code> <code>"tool_use" </code> <code>"end_turn"</code>	文本层无显式信号，需解析推断

维度	Claude tool_use 协议 (客户端侧)	Qwen 原生格式 (模型侧)
消息结构	<code>content: [block, block, ...]</code> (类型化 列表)	<code>content: "string"</code> (纯文本)

为什么不直接让客户端说 Qwen 格式 (更简单的方案)：

1. **教学映射断裂：** 学生看到 `<tool_call>` 标签和 OpenAI 函数调用结构，会以为 Claude Code 也是这么工作的。实际上 Claude Code 的客户端处理的是 `tool_use content blocks`，而非 XML 标签解析。
2. **协议设计差异是核心教学点：** Claude 的 `tool_use_id` 关联机制、`content blocks` 类型系统、`stop_reason` 信令——这些设计决策各有工程考量。如果客户端直接用 Qwen 格式，这些讨论无从展开。
3. **适配层本身是教材：** `adapter.py` 的存在让学生直观看到“API 基础设施在幕后做了什么”——同一个工具调用，从模型文本输出到 API 返回的结构化对象，中间经历了解析、验证、类型化的完整过程。

三个设计收益：

1. **客户端代码与 Claude Code 真实架构 1:1 映射：** `client.py` 中的 `stop_reason == "tool_use"` 循环判断、`content blocks` 遍历、`tool_use_id` 关联构造——每一行都能在 Claude Code 的实际行为中找到对应物。
2. **模型用原生格式推理，最大化工具调用可靠性：** Qwen 的 `<tool_call>` 格式是训练时见过的，用原生格式比强迫模型学习 Claude 格式可靠得多。
3. **适配层的存在本身解释了“为什么前沿模型不需要这一层”：** Claude 的 `tool calling` 是原生能力 + API 层结构化保障，不需要文本层解析。MVP 需要这个适配层恰恰说明了 7B 模型与前沿模型在工具调用上的架构差异——这直接呼应 5.1 节“前沿模型如何跳过防御层”的讨论。

已具备的 demo 能力：

- **协议适配架构：** 客户端说 Claude 协议，模型说原生格式，adapter 双向翻译
- 客户端-服务器架构：模型加载一次，客户端即连即用
- SSE 实时流式输出：token 逐个显示，直观展示生成过程
- 多轮工具执行：agent 最多链式执行 10 轮 (Read/Edit/Grep/Bash + Task 管理 + Agent 生成)
- 鲁棒解析器：处理小模型常见的格式畸变 (三引号、尾逗号、残缺标签)
- 任务管理系统：TaskCreate/Update/List 实现看板式协调
- Agent Team 模式：主 Agent 拆解任务，spawn 子 Agent 并行执行
- 主动探索行为：system prompt 指示模型自主发现文件，而非要求用户粘贴代码

5.3 各模块 Demo 策略

Demo 1: Chat vs Agent 对比（模块一 · 09:30 破冰）

目标： 30 秒内让观众直观感受“开环 vs 闭环”的差距。

准备： - 预录一段 Chat 模式对话：把 `buggy_code.py` 的内容粘贴给模型，模型一次性生成修复（可能正确也可能幻觉） - Live 演示 Agent 模式：只告诉 agent “tests/buggy_code.py 有 bug，帮我修”，观众看到 agent 自主 Read → 分析 → Write fix → Bash 验证

观众收获： Agent 的核心价值不是“生成能力更强”，而是“能验证自己的输出”。

Demo 2: tool_use 协议拆解（模块一 · 10:00 工具调用）

目标： 让观众看到工具调用不是黑魔法，而是可审计的 JSON 请求/响应。同时展示协议适配层如何在 Claude 格式和 Qwen 原生格式之间做双向翻译。

准备： - 并排展示两段 curl 请求：一段普通 chat completion，一段带 tools 定义的请求 - 使用 MVP 的 `model_server.py` 作为后端，手动构造带工具定义的请求，展示模型返回的 `<tool_call>` 结构 - 展示 `parser.py` 的三种解析策略：XML 标签、代码块、裸 JSON —— 说明为什么小模型需要如此宽容的解析 - **新增：** 展示 `adapter.py` 的双向转换过程——同一次工具调用在 Claude 协议和 Qwen 原生格式中的不同表达，对照 5.2a 的格式对比表

观众收获： 工具调用 = 应用层 JSON 协议，不是 tokenizer 魔法。小模型的格式不稳定恰好解释了为什么 Claude Code 选择严格 schema。适配层的存在说明了 API 基础设施在幕后做的“翻译”工作。

Demo 3: 修 Bug 全流程（模块一 · 10:50 修 Bug 实战）

目标： 完整展示一次 L → R → V 循环在 ReAct 框架下的动态执行。

场景： `tests/buggy_code.py` 中的 off-by-one 错误（`numbers[i + 1]` → `numbers[i]`）。

Live 演示步骤： 1. 启动 `model_server.py`（课前预热，模型已加载） 2. 运行 `main.py`，输入：“tests/buggy_code.py 运行会报错，帮我修复” 3. 观众观察 agent 的 TAO 循环： - **Thought：** 模型决定先读文件 → **Action：** Read tool → **Observation：** 看到代码 - **T：** 分析出 off-by-one → **A：** Write 修复 → **O：** 文件已写入 - **T：** 应该验证 →

A: Bash `python buggy_code.py` → **O:** 运行通过 4. 在每一步暂停标注：这是 L（定位）/ R（修复）/ V（验证）的哪个阶段

预备方案： 如果模型第一次格式出错或修错，**不要重启**——这正是讲 Reflexion 双层循环和 Circuit Breaker 的素材。直接标注“这就是 Orient 环节失灵”。

Demo 4: ACI 信息粒度控制（模块二 · 11:20 ACI 设计）

目标： 展示“为什么不能 cat 大文件”以及粗细粒度漏斗策略。

准备： - 准备一个较大的 Python 文件（200+ 行），让 agent 用 Read 工具读取 - 对比演示：用 Bash `cat` 读取全文（上下文爆炸，后续推理质量下降）vs 用 Read + offset/limit 精准读取（保护 token budget） - 展示 MVP 的 `client.py` 中 system prompt 如何嵌入文件树（50 行上限）—— 这就是 ACI 的信息粒度控制

观众收获： ACI 不是一个抽象概念，而是一组具体的工程决策：限制输出长度、分层漏斗、文件树快照。

Demo 5: Agentless vs Agent 对比（模块二 · 12:00 架构反思）

目标： 同一个 bug，展示两种修复路线的差异。

准备： - **Agent 路线（Live）：** 用 MVP 跑一次完整的修 Bug 流程（复用 Demo 3 的录像或再跑一次） - **Agentless 路线（PPT/伪代码）：** 展示如果用 Agentless 三阶段流水线处理同一个 bug： - L：给模型文件列表 → 选 `buggy_code.py` → 给模型代码 → 选第 8 行 - R：采样 5 个候选 patch（high temperature） - V：跑测试，选通过的那个 - 在白板上并排画出两条路径的时间线和 API 调用次数

观众收获： 简单问题两条路线殊途同归；复杂问题 agent 的动态纠偏优势才显现。

Demo 6: Agent Team 协同（模块三 · 12:20 Agent Team）

目标： 展示多 agent 拆解-并行-汇总的完整流程。

策略：Live + 预录混合。 7B 模型难以胜任 orchestrator 角色，采用分层策略： - **预录视频：** 提前用 Claude Code 录制一次完整的 Agent Team 交付过程（主 agent 拆解任务 → spawn 3 个子 agent → 并发执行 → 汇总 merge → 集成测试），作为“工业级实现”的参照 - **Live 演示：** 用 MVP + 7B 模型演示**单个 worker 的执行**——从主 agent 视角手动下发一个子任务 prompt，观众看到子 agent 在独立上下文中完成代码修改 - **对比讲解：** 7B 作为 orchestrator 的失败尝试（可选）—— 如果时间允许，现场演示 7B 尝试拆解复杂任务，观众亲眼看到它的规划能力不足，自然引出“为什么 Team 模式需要强模型”

观众收获：子 agent 的执行是可标准化的（7B 够用），orchestrator 的规划是瓶颈（需要强模型）。这与“循环机制是放大器，但基础能力是门槛”形成呼应。

5.4 Demo 准备清单

序号	准备项	用途	模块
1	启动 <code>model_server.py</code> （Qwen2.5-Coder-7B），确认 GPU 就绪	全部 Live Demo	全部
2	<code>tests/buggy_code.py</code> （off-by-one bug）	Bug 修复演示	一、二
3	<code>tests/large_module.py</code> （442 行，含 <code>ZeroDivisionError</code> ）	ACI 信息粒度对比	二
4	两段 curl 命令：普通 chat vs <code>tool_use</code> 请求	协议拆解	一
5	<code>adapter.py</code> 协议格式对比表 + 双向转换 Demo	协议适配层教学	一
6	Chat 模式修 Bug 的预录视频/截图	开环 vs 闭环对比	一
7	Claude Code Agent Team 的预录视频	Team 模式全流程	三
8	Agentless 三阶段流水线的 PPT/伪代码	架构对比	二
9	二维认知地图打印版（L-R-V × OODA）	全程导航	全部

5.5 关键教学设计原则

原则一：失败优先（Failure First）。每个 Demo 开始时预期模型可能出错。出错时不回避、不重启，而是当场标注“这是 OODA 循环的哪个环节失灵了”，把事故变成教学素材。7B 模型的能力边界是可预测的，这正是选择它的原因。

原则二：从能力边界推导工程选择。7B 能跑通模块一二的 demo，说明 ReAct 循环确实是放大器；7B 跑不了模块三的 orchestrator，说明基础能力确实是门槛。观众从亲眼所见推导出 Claude Code 选型逻辑，比讲师直接告诉答案更有说服力。

原则三：Live 演示为主，预录为辅。 模块一二全部 Live，模块三 Live + 预录混合。Live 的不确定性本身就是教学资源——它证明了 agent 的行为不是确定性脚本，而是概率性决策。

授课建议

- 1. **Boyd 故事是全场破冰利器：** 用 F-86 vs MiG-15 的故事开场，高阶开发者会立刻意识到这不是一堂 AI 营销课。从空战引出 OODA，再从 OODA 落到 ReAct，自然过渡。
 - 2. **二维认知地图要常驻屏幕：** 横轴 L-R-V × 纵轴 OODA 的论文定位图，建议打印或常驻屏幕一角，每讲完一节就标记当前位置，帮助听众始终知道“我在哪”。
 - 3. **准备 API 请求对比 Demo：** 在大屏幕上对比普通的 chat completion 请求和包含 tools 定义的请求，展示 tool_use / tool_result 的完整交互流程。
 - 4. **永远先讲失败：** 高阶开发者不相信魔法。讲每个模块时，先展示“模型在这里会犯什么错”（幻觉代码、死循环、上下文丢失），再讲工程上如何兜底。
 - 5. **用 MVP 代码做 Live Demo：** 手头的 MVP 代码可以直接演示 ReAct 循环、工具调用、错误恢复的全过程，比论文截图有说服力 10 倍。
 - 6. **双主线收束：** 结语用一句话总结——“循环转速决定胜负，接口设计决定上限，协同架构决定规模，安全机制决定底线。”
-

v3.3 | 2026-03-26 | 5.2 重写为完整 MVP 架构说明，新增协议适配层 (Adapter Pattern) 架构决策：Claude tool_use ↔ Qwen 原生格式双向转换、协议格式对比表、三个设计收益分析 v3.2 | 2026-03-25 | 新增前沿模型对比 (Claude 如何跳过防御层)、模型规模与工程重心关系、7B + 循环放大的正面叙事 v3.1 | 2026-03-25 | 5.1 新增 Dense vs MoE 技术原理分析、工业界结构化输出四层防御最佳实践 v3.0 | 2026-03-25 | 新增实践演练方案：Demo 模型选型 (Qwen2.5-Coder-7B)、各模块 Demo 策略、MVP 代码库集成、准备清单 v2.3 | 2025-03-25 | 引用格式统一为题注式，新增 Action Space 设计分析，新增 Team 模式嵌套 OODA/L-R-V 框架 v2.2 | 2025-03-25 | 引入 OODA 演进线与双主线框架、Boyd 理论、TAO 与 OODA 关系、Reflexion 双层循环 v2.0 | 2025-03-25 | 基于 Claude Code (Opus 4.6) 审校修订 v1.0 | 原始版本 by Gemini 协助生成